

Home Assignment:

Implement Playwright with TypeScript

Use Ask Mode to analyze the existing project structure and identify all impacted files required for the Playwright with TypeScript implementation.

Once the analysis is complete, switch to Agent Mode to implement the required changes

```
/**
 * Collection of default prompts for different use cases (ICE POT Format)
 */
export const DEFAULT_PROMPTS = {
  /**
   * Playwright TypeScript Page Object Prompt (No Test Class)
   */
  PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY: `
    Instructions:
    - Generate ONLY a Playwright TypeScript Page Object class (no test code).
    - Use Playwright's Locator API and import types from '@playwright/test' only when needed.
    - Prefer stable locators such as getByRole, getByLabel, getByText, and test ids from the DOM.
    - Add typed properties, a constructor that accepts Page, and meaningful async methods.
    - Add concise JSDoc for the class and public methods.
    - Do NOT include explanations, feature files, or test code.

    Context:
    DOM:
    \`\`\`html
    \${domContent}
    \`\`\`
```

Example:

```
\\\`typescript
```

```
import { Page } from '@playwright/test';
```

```
/**
```

```
 * Page Object for Component Page.
```

```
 */
```

```
export class ComponentPage {
```

```
  constructor(private readonly page: Page) {}
```

```
  /** Opens the component page. */
```

```
  async open(): Promise<void> {
```

```
    await this.page.goto(`${pageUrl}`);
```

```
  }
```

```
}
```

```
\\\`
```

Persona:

- Audience: TypeScript automation engineers building maintainable Playwright page objects.

Output Format:

- A single TypeScript class inside a `\\\`typescript\\\`` block.

Tone:

- Clean, typed, maintainable, enterprise-ready.

```
`
```

```
/**
```

```
 * Cucumber feature with Playwright TypeScript step definitions
```

```
 */
```

CUCUMBER_WITH_PLAYWRIGHT_TYPESCRIPT_STEPS: `

Instructions:

- Generate BOTH:

1. A valid Cucumber .feature file.
2. A TypeScript step definition class or module using Playwright.

- Do NOT include Java, Selenium, or Page Object code.

- Step definitions must include browser/page setup and teardown, explicit Playwright actions, and assertions.

- Use async/await and imports from '@playwright/test' where appropriate.

- Use Scenario Outline with an Examples table containing South India realistic data where applicable.

- Every step must be relevant to the provided DOM and should use stable locators.

Context:

DOM:

\\`html

\\\${domContent}

\\`

URL: \\\${pageUrl}

Example:

\\`gherkin

Feature: Login

Scenario Outline: Successful login with valid credentials

Given I open the login page

When I enter "<username>" in the Username field

And I enter "<password>" in the Password field

And I click the Login button

Then I should see the dashboard

Examples:

```
| username | password |
```

```
| admin   | admin123 |
```

```
\\`
```

```
\\`typescript
```

```
import { chromium, expect, type Browser, type Page } from '@playwright/test';
```

```
import { Given, When, Then, Before, After } from '@cucumber/cucumber';
```

```
let browser: Browser;
```

```
let page: Page;
```

```
Before(async function () {
```

```
  browser = await chromium.launch();
```

```
  page = await browser.newPage();
```

```
});
```

```
After(async function () {
```

```
  await browser.close();
```

```
});
```

```
Given('I open the login page', async function () {
```

```
  await page.goto(`\${pageUrl}`);
```

```
});
```

```
Then('I should see the dashboard', async function () {
```

```
  await expect(page.getByRole('heading', { name: /dashboard/i })).toBeVisible();
```

```
});
```

```
\\`
```

Persona:

- Audience: QA engineers using Cucumber, Playwright, and TypeScript.

Output Format:

- Gherkin in a ````gherkin```` block followed by TypeScript in a ````typescript```` block.

Tone:

- Professional, executable, typed, structured.

,

/**

* Selenium Java Page Object Prompt (No Test Class)

*/

SELENIUM_JAVA_PAGE_ONLY: `

Instructions:

- Generate ONLY a Selenium Java Page Object Class (no test code).
- Add JavaDoc for methods & class.
- Use Selenium 2.30+ compatible imports.
- Use meaningful method names.
- Do NOT include explanations or test code.

Context:

DOM:

````html`

`\${domContent}`

`````

Example:

````java`

`package com.testleaf.pages;`

/\*\*

\* Page Object for Component Page

```
*/

public class ComponentPage {
 // Add methods as per the DOM
}
\\\`
```

Persona:

- Audience: Automation engineer focusing on maintainable POM structure.

Output Format:

- A single Java class inside a `\\\`java\\\` block.

Tone:

- Clean, maintainable, enterprise-ready.

`,

```
/**
```

\* Cucumber Feature File Only Prompt

```
*/
```

CUCUMBER\_ONLY: `

Instructions:

- Generate ONLY a Cucumber (.feature) file.
- Use Scenario Outline with Examples table.
- Make sure every step is relevant to the provided DOM.
- Do not combine multiple actions into one step.
- Use South India realistic dataset (names, addresses, pin codes, mobile numbers).
- Use dropdown values only from provided DOM.
- Generate multiple scenarios if applicable.

Context:

DOM:

\\\`html

\\${domContent}

\\\`

Example:

\\\`gherkin

Feature: Login to OpenTaps

Scenario Outline: Successful login with valid credentials

Given I open the login page

When I type "<username>" into the Username field

And I type "<password>" into the Password field

And I click the Login button

Then I should be logged in successfully

Examples:

| username | password |

| "testuser" | "testpass" |

| "admin" | "admin123" |

\\\`

Persona:

- Audience: BDD testers who only need feature files.

Output Format:

- Only valid Gherkin in a `\\\`gherkin\\\` block.

Tone:

- Clear, structured, executable.

,

/\*\*

\* Cucumber with Step Definitions

\*/

CUCUMBER\_WITH\_SELENIUM\_JAVA\_STEPS: `

Instructions:

- Generate BOTH:

1. A Cucumber .feature file.
2. A Java step definition class for selenium.

- Do NOT include Page Object code.

- Step defs must include WebDriver setup, explicit waits, and actual Selenium code.

- Use Scenario Outline with Examples table (South India realistic data).

Context:

DOM:

\\`html

\\${domContent}

\\`

URL: \\${pageUrl}

Example:

\\`gherkin

Feature: Login to OpenTaps

Scenario Outline: Successful login with valid credentials

Given I open the login page

When I type "<username>" into the Username field

And I type "<password>" into the Password field

And I click the Login button

Then I should be logged in successfully

Examples:



```
 | username | password |
 \ | "admin" | "admin123" |
 \ \ \`
```

```
\ \ \ `java
```

```
package com.leaf taps.stepdefs;
```

```
import io.cucumber.java.en.*;
```

```
import org.openqa.selenium.*;
```

```
import org.openqa.selenium.chrome.ChromeDriver;
```

```
import org.openqa.selenium.support.ui.*;
```

```
public class LoginStepDefinitions {
```

```
 private WebDriver driver;
```

```
 private WebDriverWait wait;
```

```
@io.cucumber.java.Before
```

```
public void setUp() {
```

```
 driver = new ChromeDriver();
```

```
 wait = new WebDriverWait(driver, Duration.ofSeconds(10));
```

```
 driver.manage().window().maximize();
```

```
}
```

```
@io.cucumber.java.After
```

```
public void tearDown() {
```

```
 if (driver != null) driver.quit();
```

```
}
```

```
@Given("I open the login page")
```

```
public void openLoginPage() {
```

```
 driver.get("${page Url}");
```

```
}
```

```
@When("I type {string} into the Username field")
```

```
public void enterUsername(String username) {
```

```
 WebElement el = wait.until(ExpectedConditions.elementToBeClickable(By.id("username")));
```

```
 el.sendKeys(username);
```

```
}
```

```
@When("I type {string} into the Password field")
```

```
public void enterPassword(String password) {
```

```
 WebElement el = wait.until(ExpectedConditions.elementToBeClickable(By.id("password")));
```

```
 el.sendKeys(password);
```

```
}
```

```
@When("I click the Login button")
```

```
public void clickLogin() {
```

```
 driver.findElement(By.xpath("//button[contains(text(),'Login')]")).click();
```

```
}
```

```
@Then("I should be logged in successfully")
```

```
public void verifyLogin() {
```

```
 WebElement success =
```

```
wait.until(ExpectedConditions.visibilityOfElementLocated(By.className("success")));
```

```
 assert success.isDisplayed();
```

```
}
```

```
}
```

```
\\\`
```

Persona:

- Audience: QA engineers working with Cucumber & Selenium.

Output Format:

- Gherkin in ````gherkin```` block + Java code in ````java```` block.

Tone:

- Professional, executable, structured.

```

`
};

/**
 * Helper function to escape code blocks in prompts
 */
function escapeCodeBlocks(text) {
 return text.replace(/`/g, '`');
}

/**
 * Function to fill template variables in a prompt
 */
export function getPrompt(promptKey, variables = {}) {
 let prompt = DEFAULT_PROMPTS[promptKey];
 if (!prompt) {
 throw new Error(`Prompt not found: ${promptKey}`);
 }

 Object.entries(variables).forEach(([k, v]) => {
 const regex = new RegExp(`\\$\\{${k}\\}`, 'g');
 prompt = prompt.replace(regex, v);
 });

 return prompt.trim();
}
```

```
export const CODE_GENERATOR_TYPES = {
 SELENIUM_JAVA_PAGE_ONLY: 'Selenium-Java-Page-Only',
 PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY: 'Playwright-TypeScript-Page-Only',
 CUCUMBER_ONLY: 'Cucumber-Only',
 CUCUMBER_WITH_SELENIUM_JAVA_STEPS: 'Cucumber-With-Selenium-Java-Steps',
 CUCUMBER_WITH_PLAYWRIGHT_TYPESCRIPT_STEPS: 'Cucumber-With-Playwright-TypeScript-Steps',
};
```